

团队协作操作指南 / 팀 협업 가이드

本文档分为两部分：**第一部分是完整中文版**，**第二部分是完整韩文版**，两部分内容完全对应，挑你需要的语言看就行，不用两边来回对照。

이 문서는 두 부분으로 구성되어 있습니다: **1부는 완전한 중국어판**, **2부는 완전한 한국어판**이며 두 부분의 내용은 서로 동일합니다. 필요한 언어만 보시면 되고, 두 언어를 번갈아 비교하며 읽을 필요는 없습니다.

第一部分：中文版

目录

0. 你需要准备什么
 1. 几个概念（先看这个）
 2. 第一次设置（只做一次）
 3. 日常协作流程（每次都这样做）
 4. 不想用命令行？用 GitHub Desktop
 5. ⚠ 本项目的特别注意事项（务必完整看完）
-

0. 你需要准备什么

1. 一个 GitHub 账号（github.com 免费注册）。
 2. 把你的 GitHub 用户名发给项目负责人（仓库拥有者），让 TA 在仓库的 **Settings → Collaborators** 里把你添加为协作者。你会收到一封邮件邀请，点击接受。
 3. 在电脑上安装 **Git**：
 - Windows: <https://git-scm.com/download/win>, 一路下一步安装即可。
 - Mac: 终端输入 `git --version`, 如果没装会提示自动安装。
 4. （可选，但强烈推荐新手用）安装 **GitHub Desktop**（图形界面，不用敲命令）：
<https://desktop.github.com/>
-

1. 几个概念（先看这个）

术语	大白话解释
仓库 Repository / repo	这个项目的文件 + 修改历史，存在 GitHub 上的一个"文件夹"
克隆 Clone	把 GitHub 上的仓库完整下载一份到你自己的电脑上
分支 Branch	一条独立的"时间线"。 <code>main</code> 分支是正式、稳定的版本；每个人开发新功能时，应该开一条 自己的分支 ，改完确认没问题再合并回 <code>main</code> ，这样不会互相覆盖
提交 Commit	把你改的内容"存一个存档点"，并写一句话说明"这次改了"
推送 Push	把你电脑上的存档点上传到 GitHub
拉取 Pull	把 GitHub 上别人（或你自己之前）的修改下载到电脑上，保持同步
Pull Request (PR)	你在 GitHub 网页上发起的一个"申请"："我这条分支改好了，请把它合并到 main"。队友可以在 PR 页面看你改了、留评论，负责人确认没问题后点击合并

一句话总结工作方式：永远不要直接在 `main` 分支上改东西。每次开发前先拉一条自己的分支，改完发 Pull Request，让别人看过、没问题再合并。这样即使你改错了，`main` 分支上线上能跑的版本也不会被破坏。

2. 第一次设置（只做一次）

2.1 告诉 Git 你是谁

打开终端（Windows 用 Git Bash，Mac 用「终端」App），输入：

```
git config --global user.name "你的名字"
git config --global user.email "你的GitHub注册邮箱"
```

2.2 克隆仓库到你的电脑

先在网页上打开仓库地址 `https://github.com/ChenYvhang/glimmer-scout`，点击绿色的 **Code** 按钮，复制 HTTPS 链接。然后在终端里，`cd` 到你放项目的文件夹，输入：

```
git clone https://github.com/ChenYvhang/glimmer-scout.git
cd glimmer-scout
```

第一次 push 时，GitHub 会要求你登录——不能用密码，要用 **Personal Access Token**（相当于密码的替代品）。生成方法：GitHub 网页右上角头像 → **Settings** → 左侧最下方 **Developer settings** → **Personal access tokens** → **Tokens (classic)** → **Generate new token**，勾选 `repo` 权限，生成后**立刻复制保存**（只显示一次）。之后 Git 弹出登录框时，用户名填 GitHub 用户名，密码填这个 token。

3. 日常协作流程（每次都这样做）

第①步：开始工作前，先同步最新代码

```
git checkout main
git pull origin main
```

第②步：给自己开一条新分支

分支名建议格式：**你的名字/这次做的事**，比如 `xiaoming/fix-scatter-tooltip`。

```
git checkout -b xiaoming/fix-scatter-tooltip
```

第③步：正常修改文件

用你熟悉的编辑器（VS Code 等）改代码就行，跟平时写代码没区别。

第④步：把修改保存成一个“存档点”

```
git status          # 看看改了哪些文件
git add 具体改过的文件名 # 例如：git add web/src/App.tsx
git commit -m "一句话说明改了什么" # 例如："修复散点图悬停提示不消失的问题"
```

⚠️ **不要用 `git add .` 或 `git add -A`**（会把所有文件不加区分地打包提交，容易误提交不该提交的东西，比如密钥文件）。养成“逐个文件确认”的习惯。

第⑤步：推送到 GitHub

```
git push origin xiaoming/fix-scatter-tooltip
```

第一次推送这条分支时，终端会提示你复制一行带 `--set-upstream` 的命令，照着执行一次即可（以后同一条分支直接 `git push` 就行）。

第⑥步：在 GitHub 网页上发起 Pull Request

推送成功后，打开仓库网页 <https://github.com/ChenYvhang/glimmer-scout>，通常会自动弹出一个黄色提示条“Compare & pull request”，点它。填写标题和简单说明（改了什么、为什么改），确认 base 分支是 `main`，点击 **Create pull request**。

第⑦步：等待 review、合并

其他人（或项目负责人）会在 PR 页面看你的改动、留评论。确认没问题后，点击 **Merge pull request** 合并进 `main`。合并完可以点击 **Delete branch** 删掉这条临时分支（不影响你电脑上的代码）。

之后回到“第①步”，继续下一个任务。

💡 **备注：**以上这些日常操作（拉取、开分支、提交、推送、发 PR）也完全可以让 AI 编程助手（如 Claude Code）帮你代劳——把“我要做什么”告诉它，让它执行对应的 git 命令即可，你只需要确认结果，不必自己

一条条敲命令。

4. 不想用命令行？用 GitHub Desktop

如果敲命令让你紧张，**GitHub Desktop** 是一个图形界面工具，上面第 3 节的每一步都有对应的 按钮，不用记命令：

命令行操作	GitHub Desktop 里对应的按钮
<code>git clone</code>	打开 App → File → Clone Repository
<code>git pull</code>	顶部 Fetch origin / Pull origin 按钮
<code>git checkout -b</code>	顶部 Current Branch 下拉框 → New Branch
<code>git add</code> + <code>git commit</code>	左侧勾选改动的文件 → 下方填提交说明 → Commit to 分支名
<code>git push</code>	顶部 Push origin 按钮
发起 Pull Request	push 后点顶部 Create Pull Request，会自动跳转到浏览器

备注：如果你用的是能直接操作终端/执行命令的 AI 工具（如 Claude Code），上面这些 按钮操作同样可以让它代劳，不需要在命令行和 GitHub Desktop 之间选一个自己学——直接让 AI 帮你做，你负责确认结果即可。

! 5. 本项目的特别注意事项（务必完整看完）

- .env 文件绝对不要提交**（也不需要提交）。它里面是 API 密钥（YouTube/智谱/DeepSeek），一旦推到公开仓库上会被恶意盗用。这个文件已经写进 `.gitignore`，正常操作 不会提交它，但如果 `git status` 里看到 `.env` 出现在改动列表里，**不要 add/commit 它**，先来问一下项目负责人。
- web/public/dataset.json 是自动生成的文件，不要手改**。它是跑 `python -m pipeline.build` 之后生成的，如果你是负责数据管道（pipeline）那部分的人，改完 `pipeline/` 里的代码后重新跑一次 build 再提交这个文件；如果你是只负责前端页面的人，一般不需要动这个文件。
- 这个文件很大（约 60MB）**，`git push` 可能要几十秒到几分钟，看到进度条转着不要以为卡住了。GitHub 会提示“文件超过 50MB 建议用 Git LFS”，这是警告不是错误，可以先不用管，push 依然会成功。
- pipeline/cache/、pipeline/artifacts/、pipeline/raw/、.venv/、web/node_modules/、web/dist/ 都已经 在 .gitignore 里**，`git status` 不会看到它们，不用担心误提交。
- 分工建议：**负责 `pipeline/` Python 代码的人和负责 `web/` 前端代码的人，改动的文件 基本不重叠，冲突概率很低。如果两人都要改同一个文件（比如 `web/src/lib/schema.ts`），提前在群里说一声，谁先改完先发 PR、合并后另一人再拉最新代码。

6. **不要用 `git push --force`** (强制推送)。这会覆盖别人在远程仓库上的提交, 可能导致 队友的工作丢失。
如果 push 失败提示"rejected", 先 `git pull` 同步别人的最新改动, 解决完冲突再 push。
 7. **本地用 AI (如 Claude Code) 辅助改代码时, 一定要求它阅读全部源代码, 不要只让它看 `README.md` / `PLAN.md` 就动手改。** README 和 PLAN 是给人看的摘要, 省略了大量实现 细节; AI 如果偷懒只扫一眼这两份文档就直接改代码, 很容易改出和现有实现方式不一致、 甚至悄悄覆盖掉已有设计的东西, 而且不会主动提醒你它没有全部读过。**哪怕要求"完整 阅读所有源代码"会明显更慢、消耗大量 token, 也必须这样要求——这不是可以为了省事 跳过的步骤。**
 8. **每次让 AI 改代码前, 先让它把这一轮打算做的改动写进 `PLAN.md`, 确认方案没问题后 再动手, 不要边想边改、改完才补记录。** 这样每一次迭代都留下清晰的规划和执行痕迹: 队友事后能看懂某个改动的来龙去脉, 下一次协作时也能先看 `PLAN.md` 就了解当前进展, 不用去猜或重新问一遍 AI。
-
-

제2부: 한국어판

목차

- 0. 준비물
 - 1. 몇 가지 개념 (먼저 읽어보세요)
 - 2. 최초 설정 (한 번만 하면 됩니다)
 - 3. 일상적인 협업 흐름 (매번 이렇게 하세요)
 - 4. 명령어가 부담스럽다면 GitHub Desktop
 - 5. ⚠ 이 프로젝트의 특별 주의사항 (반드시 끝까지 읽으세요)
-

0. 준비물

- 1. GitHub 계정 하나 (github.com 에서 무료 가입).
 - 2. 본인의 GitHub 아이디를 프로젝트 담당자(저장소 소유자)에게 알려주고, **Settings → Collaborators** 메뉴에서 협업자로 추가해달라고 요청하세요. 초대 이메일이 오면 수락(Accept)하면 됩니다.
 - 3. 컴퓨터에 **Git** 설치:
 - Windows: <https://git-scm.com/download/win> 에서 다운로드 후 계속 다음(Next)만 누르면 됩니다.
 - Mac: 터미널에 `git --version` 입력 시 설치 안내가 뜨면 그대로 설치하세요.
 - 4. (선택이지만 초보자에게 강력 추천) **GitHub Desktop** 설치 (명령어 없이 그래픽으로 조작):
<https://desktop.github.com/>
-

1. 몇 가지 개념 (먼저 읽어보세요)

용어	쉬운 설명
저장소 Repository / repo	이 프로젝트의 모든 파일 + 수정 이력이 저장된, GitHub 상의 "폴더"
클론 Clone	GitHub에 있는 저장소 전체를 내 컴퓨터로 그대로 다운로드하는 것
브랜치 Branch	독립된 "타임라인" 하나. <code>main</code> 브랜치는 정식으로 배포되는 안정 버전이고, 새 기능을 개발할 때는 자기만의 브랜치 를 만들어서 작업한 뒤, 문제없는지 확인하고 <code>main</code> 에 합치는 것이 원칙입니다. 이렇게 하면 서로의 작업을 덮어쓰지 않습니다
커밋 Commit	수정한 내용을 "저장 포인트"로 남기고, "이번에 무엇을 바꿨는지" 한 줄로 기록하는 것
푸시 Push	내 컴퓨터에 있는 저장 포인트를 GitHub에 업로드하는 것
풀 Pull	GitHub에 있는 다른 사람(또는 내가 이전에 올린) 수정 내용을 내 컴퓨터로 다운로드해서 동기화하는 것
풀 리퀘스트 Pull Request (PR)	GitHub 웹페이지에서 올리는 일종의 "신청서": "제 브랜치에서 작업을 마쳤으니 main에 합쳐주세요." 팀원들이 PR 페이지에서 변경 내용을 보고 댓글을 남길 수 있고, 담당자가 문제없다고 확인하면 병합(merge)합니다

한 줄 요약: `main` 브랜치에서 절대 직접 작업하지 마세요. 작업을 시작하기 전에 항상 자기만의 브랜치를 새로 만들고, 작업이 끝나면 Pull Request를 올려서 다른 사람이 검토한 뒤 병합하도록 하세요. 이렇게 하면 실수로 코드를 망가뜨려도 실제 배포되는 `main` 브랜치는 안전합니다.

2. 최초 설정 (한 번만 하면 됩니다)

2.1 Git에게 내가 누구인지 알려주기

터미널을 열고 (Windows는 Git Bash, Mac은 「터미널」 앱), 아래를 입력하세요:

```
git config --global user.name "이름"
git config --global user.email "GitHub 가입 이메일"
```

2.2 저장소를 내 컴퓨터로 클론하기

먼저 웹브라우저에서 저장소 주소 <https://github.com/ChenYvhang/glimmer-scout> 를 열고, 초록색 **Code** 버튼을 눌러 HTTPS 링크를 복사하세요. 그다음 터미널에서 프로젝트를 저장할 폴더로 `cd` 이동한 뒤 다음을 입력합니다:

```
git clone https://github.com/ChenYvhang/glimmer-scout.git
cd glimmer-scout
```

첫 push를 할 때 GitHub이 로그인을 요구하는데, 비밀번호가 아니라 **Personal Access Token**(비밀번호 대체 토큰)을 사용해야 합니다. 생성 방법: GitHub 우측 상단 프로필 → **Settings** → 왼쪽 맨 아래 **Developer settings** → **Personal access tokens** → **Tokens (classic)** → **Generate new token** 에서 **repo** 권한을

체크하고 생성한 뒤 **바로 복사해서 저장**하세요 (한 번만 보여줍니다). 이후 Git이 로그인 창을 띄우면 사용자 이름에는 GitHub 아이디, 비밀번호 칸에는 이 토큰을 입력하면 됩니다.

3. 일상적인 협업 흐름 (매번 이렇게 하세요)

① 작업 시작 전, 최신 코드로 동기화

```
git checkout main
git pull origin main
```

② 나만의 새 브랜치 만들기

브랜치 이름 형식 추천: **이름/작업내용**, 예: `xiaoming/fix-scatter-tooltip`.

```
git checkout -b xiaoming/fix-scatter-tooltip
```

③ 평소대로 파일 수정하기

평소 사용하던 에디터(VS Code 등)로 코드를 수정하면 됩니다. 평소와 다를 게 없습니다.

④ 수정 내용을 "저장 포인트"로 만들기

```
git status          # 어떤 파일이 바뀌었는지 확인
git add 구체적으로 수정한 파일명      # 예: git add web/src/App.tsx
git commit -m "이번에 무엇을 바꿨는지 한 줄 설명"  # 예: "산점도 호버 툴팁이 사라지지 않는 문제 수정"
```

⚠ **git add .** 나 **git add -A** 는 사용하지 마세요 (모든 파일을 구분 없이 한꺼번에 커밋하게 되어, 키 파일 같은 것을 실수로 커밋할 위험이 있습니다). 파일을 하나씩 확인하고 추가하는 습관을 들이세요.

⑤ GitHub에 푸시하기

```
git push origin xiaoming/fix-scatter-tooltip
```

이 브랜치를 처음 푸시할 때, 터미널에 `--set-upstream` 이 포함된 명령어를 복사해서 실행하라는 안내가 뜹니다. 한 번만 그대로 실행하면 됩니다 (이후에는 같은 브랜치에서 그냥 `git push` 만 하면 됩니다).

⑥ GitHub 웹에서 Pull Request 만들기

푸시가 성공하면 저장소 웹페이지 <https://github.com/ChenYvhang/glimmer-scout> 를 여세요. 보통 노란색 "Compare & pull request" 알림이 자동으로 뜨는데, 그걸 클릭하세요. 제목과 간단한 설명(무엇을 바꿨는지, 왜 바꿨는지)을 작성하고, base 브랜치가 `main` 인지 확인한 뒤 **Create pull request**를 클릭합니다.

⑦ 리뷰를 기다리고 병합하기

다른 사람(또는 프로젝트 담당자)이 PR 페이지에서 변경 사항을 확인하고 댓글을 남길 수 있습니다. 문제없다고 확인되면 **Merge pull request**를 눌러 **main**에 병합합니다. 병합 후에는 **Delete branch**를 눌러 임시 브랜치를 지워도 됩니다 (내 컴퓨터의 코드에는 영향 없습니다).

이후 다시 "① 단계"로 돌아가서 다음 작업을 시작하면 됩니다.

참고: 위의 일상적인 작업들(pull, 브랜치 생성, 커밋, 푸시, PR 올리기)은 AI 코딩 어시스턴트(예: Claude Code)에게 그대로 맡길 수도 있습니다. "무엇을 하고 싶은지"만 알려주면 AI가 해당 git 명령을 대신 실행해 줄 수 있으니, 명령어를 하나하나 직접 칠 필요 없이 결과만 확인하면 됩니다.

4. 명령어가 부담스럽다면 GitHub Desktop

명령어가 부담스럽다면, **GitHub Desktop**은 위 3번 항목의 모든 단계를 버튼 클릭으로 대신할 수 있는 그래픽 도구입니다:

명령어 작업	GitHub Desktop에서 해당하는 버튼
<code>git clone</code>	앱 실행 → File → Clone Repository
<code>git pull</code>	상단의 Fetch origin / Pull origin 버튼
<code>git checkout -b</code>	상단 Current Branch 드롭다운 → New Branch
<code>git add</code> + <code>git commit</code>	왼쪽에서 변경된 파일 체크 → 아래에 커밋 메시지 입력 → Commit to 브랜치이름
<code>git push</code>	상단의 Push origin 버튼
Pull Request 만들기	push 후 상단의 Create Pull Request 클릭 시 브라우저로 자동 이동

참고: 터미널/명령어를 직접 실행할 수 있는 AI 도구(예: Claude Code)를 사용한다면, 위의 버튼 조작들도 마찬가지로 AI에게 맡길 수 있습니다. 명령줄과 GitHub Desktop 중 하나를 따로 배울 필요 없이, AI에게 시키고 결과만 확인하면 됩니다.



5. 이 프로젝트의 특별 주의사항 (반드시 끝까지 읽으세요)

- .env** 파일은 절대 커밋하지 마세요 (커밋할 필요도 없습니다). 이 파일에는 API 키 (YouTube/Zhipu/DeepSeek)가 들어있어서, 공개 저장소에 올라가면 악용될 수 있습니다. 이 파일은 이미 **.gitignore**에 등록되어 있어 정상적으로는 커밋되지 않지만, 만약 `git status`에서 **.env**가 변경 목록에 보인다면 **절대 add/commit 하지 말고** 먼저 프로젝트 담당자에게 문의하세요.
- web/public/dataset.json**은 자동 생성 파일이므로 직접 수정하지 마세요. 이 파일은 `python -m pipeline.build`를 실행하면 생성됩니다. 데이터 파이프라인(pipeline)을 담당하는 사람이라면 **pipeline/** 코드를 수정한 뒤 build를 다시 실행하고 이 파일을 커밋하면 되고, 프론트엔드만 담당하는 사람이라면 보통 이 파일을 건드릴 필요가 없습니다.

3. 이 파일은 용량이 큼니다 (약 60MB), `git push` 에 수십 초에서 몇 분이 걸릴 수 있으니 진행 바가 오래 돌아도 멈춘 게 아닙니다. GitHub이 "파일이 50MB를 넘으니 Git LFS를 사용하라"는 경고를 띄우는데, 이건 경고일 뿐 오류가 아니며 무시해도 push는 정상적으로 성공합니다.
4. `pipeline/cache/` , `pipeline/artifacts/` , `pipeline/raw/` , `.venv/` , `web/node_modules/` , `web/dist/` 는 모두 `.gitignore` 에 등록되어 있습니다. `git status` 에 보이지 않으니 실수로 커밋될 걱정은 안 하셔도 됩니다.
5. 역할 분담 제안: `pipeline/` Python 코드 담당자와 `web/` 프론트엔드 담당자는 수정하는 파일이 거의 겹치지 않아 충돌 가능성이 낮습니다. 만약 두 사람이 같은 파일(예: `web/src/lib/schema.ts`)을 수정해야 한다면, 미리 채팅방에서 이야기하고 먼저 끝낸 사람이 PR을 올려 병합한 뒤 다른 사람이 최신 코드를 pull 받으세요.
6. `git push --force` (강제 푸시)는 사용하지 마세요. 다른 사람이 원격 저장소에 올린 커밋을 덮어써서 팀원의 작업이 사라질 수 있습니다. push가 "rejected"라고 실패하면, 먼저 `git pull` 로 다른 사람의 최신 변경 사항을 받아온 뒤 충돌을 해결하고 다시 push하세요.
7. 로컬에서 AI(예: Claude Code)로 코드 수정을 도움받을 때는, 반드시 전체 소스 코드를 읽도록 요구하세요. `README.md` / `PLAN.md` 만 보고 바로 수정하게 하면 안 됩니다. README와 PLAN은 사람이 보기 위한 요약본이라 구현 세부사항이 많이 생략되어 있습니다. AI가 게을러서 이 두 문서만 훑어보고 바로 코드를 수정하면, 기존 구현 방식과 어긋나거나 이미 있는 설계를 조용히 덮어써버리기 쉽고, 그러고도 전체를 다 읽지 않았다는 사실을 스스로 알려주지 않습니다. "전체 소스 코드를 다 읽어라"라고 요구하면 훨씬 느려지고 토큰을 많이 쓰더라도, 반드시 그렇게 요구하세요 — 이건 편하자고 건너뛴 수 있는 단계가 아닙니다.
8. AI에게 코드 수정을 시키기 전에, 이번에 하려는 작업 내용을 먼저 `PLAN.md` 에 적게 하고, 방향을 확인한 뒤에 실제 수정을 시작하도록 하세요. 생각하면서 동시에 고치고 나중에 기록을 보완하게 하지 마세요. 이렇게 하면 매번의 작업마다 명확한 계획과 실행 흔적이 남아서, 나중에 팀원이 특정 변경의 배경을 쉽게 확인할 수 있고, 다음에 협업할 때도 `PLAN.md` 만 보면 현재 진행 상황을 바로 파악할 수 있어 AI에게 다시 물어보거나 추측할 필요가 없습니다.